Table des matières

Cours 02 — Ranger les fichiers en bronze 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Le dépôt `01_de` · RAW 2
Étape 2 — Écrire `sql/bronze.sql` · RAW 2
Étape 3 — Ranger le mois · RAW 4
Étape 4 — Contrôler le bronze contre la source · RAW 5
Étape 5 — Ce que le format change · RAW 6
Étape 6 — Versionner · RAW 7
À retenir 7

Cours 02 — Ranger les fichiers en bronze

Cours 2/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
		✓ ici

Ce que vous allez faire

Le cours 01 a trouvé deux défauts dans la caisse : des montants que DuckDB devine en `DOUBLE`, et le 12 juin exporté deux fois. Le bronze est la première couche du lac : les fichiers livrés, rangés dans un format fait pour être lu, avec des types déclarés plutôt que devinés. Vous écrivez `sql/bronze.sql`, qui range les tickets du mois en Parquet, un dossier par jour, en ne gardant d'un jour exporté deux fois que son dernier export. Puis vous le contrôlez contre la source, et vous regardez ce que le format a changé.

les exports de la caisse, dans `sources/` → `sql/bronze.sql` → `plateforme/lac/bronze/tickets/`, un dossier Parquet par jour → le contrôle contre la source

Le code complet est dans `Codes_02_DE.md`, à ouvrir après avoir essayé.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Une seule fenêtre de l'Anaconda Prompt sert à ce cours, placée dans `Projet_Brulerie`, le dossier que le cours 00 a créé : le notebook de DuckDB s'y ouvre à l'étape 4 et s'y ferme avant la dernière.

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
dir /b
REM Attendu : sources, plus ce qu'Analytics Engineering y a laissé, si vous l'avez fait.
```

Étape 1 — Le dépôt `01_de` · RAW

Le code du pipeline vit dans un dépôt Git, `01_de`, à côté des sources. Le lac, lui, reste hors du dépôt, dans `Projet_Brulerie\plateforme\lac` : Git garde le code, pas les données. `plateforme` est le dossier de ce que l'équipe de données fait tourner : le lac à ce cours, l'entrepôt au cours 03.

1.1 Créer le dépôt, puis son `.gitignore`.

```
git init 01_de
REM Attendu : Initialized empty Git repository in …/Projet_Brulerie/01_de/.git/
notepad 01_de\.gitignore
```

Le Bloc-notes propose de créer le fichier : accepter. Coller, enregistrer, fermer :

```
# Fichiers compilés par Python
__pycache__/
```

Python y rangera ses fichiers compilés dès qu'un script en importera un autre, au cours 05.

Étape 2 — Écrire `sql/bronze.sql` · RAW

Le bronze garde ce que la caisse a livré, sans rien interpréter, mais dans un format qu'on lit vite : Parquet range les valeurs colonne par colonne, les compresse, et garde dans le fichier le type de chacune. Trois décisions font le bronze :

- les types se déclarent, jamais ne se devinent : chaque colonne se nomme avec son type, et un montant se déclare en `DECIMAL`, qui compte au cent près ; un code ou un numéro de membre reste du texte ;
- un dossier par jour livré, `jour=2025-06-01`, comme dans un lac d'entreprise : le chemin porte le jour, et le jour revient comme une colonne à la lecture ;
- d'un jour exporté deux fois, le dernier export fait foi : c'est le premier constat du cours 01.

© 2026 iAlexMG. Tous droits réservés.

Le bronze se refait en entier à chaque passage : l'option `OVERWRITE` vide le dossier avant d'écrire, et relancer le script redonne le même bronze. Les noms des exports d'un même jour ne diffèrent que par l'instant de l'export, `AAAAMMJJHHMM`, à la fin : le plus grand nom, dans l'ordre alphabétique, est donc le dernier export.

Pour remplir	**L'outil**
déclarer les types d'un CSV	`columns = {'ticket_id': 'INTEGER', …}` : une paire par colonne, les treize du dictionnaire, dans l'ordre du fichier
un montant au cent	`DECIMAL(10,2)` : dix chiffres, dont deux après la virgule, sans approximation
le jour livré	le `REGEXP_EXTRACT` de l'étape 3.2 du cours 01, converti en date par `CAST(… AS DATE)`
le dernier export d'un jour	`MAX(filename) OVER (PARTITION BY jour)` rend, sur chaque ligne, le plus grand nom de fichier de son jour ; `QUALIFY` ne garde que les lignes qui portent ce nom
écrire du Parquet, un dossier par jour	`COPY (requête) TO 'dossier' (FORMAT parquet, PARTITION_BY (jour), OVERWRITE)`

`MAX(…) OVER (…)` est une fonction de fenêtre : un agrégat calculé pour chaque ligne, sur les lignes de sa partition, sans les regrouper. Elle s'apprend au Cours 04 de la formation SQL avancé, `Theorie_04_SQL.md`, sections 2 et 7, `QUALIFY` compris. `COPY` crée le dernier dossier du chemin, `tickets`, mais pas ceux du dessus : `plateforme\lac\bronze` se crée avant, à l'étape 3.

2.1 Créer le dossier des scripts SQL, puis le fichier.

```
mkdir 01_de\sql
notepad 01_de\sql\bronze.sql
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
-- 01_de/sql/bronze.sql : les tickets de la caisse en Parquet, typés, un dossier par jour livré
COPY (
    -- 1. chaque export avec son chemin, les types déclarés, les montants en DECIMAL ; le jour livré
    SELECT *, CAST(REGEXP_EXTRACT(filename, '____', 1) AS DATE) AS jour
    FROM read_csv('sources/caisse/*/tickets_*.csv', filename = true, columns = {
        'ticket_id': 'INTEGER', 'horodatage': 'TIMESTAMP', 'type_ticket': 'VARCHAR',
        'statut': 'VARCHAR', 'ticket_origine': 'INTEGER', 'membre_id': 'VARCHAR',
        'sous_total': '____', 'tps': 'DECIMAL(10,2)', 'tvq': 'DECIMAL(10,2)',
        'total': 'DECIMAL(10,2)', 'mode_paiement': 'VARCHAR', 'arrondi': 'DECIMAL(10,2)',
        'encaisse': 'DECIMAL(10,2)'})
    -- 2. d'un jour exporté deux fois, ne garder que le dernier export : il fait foi
```


© 2026 iAlexMG. Tous droits réservés.

```
    QUALIFY filename = ____(filename) OVER (PARTITION BY ____)
-- 3. un dossier par jour livré ; OVERWRITE vide le bronze, qui se refait en entier
) TO 'plateforme/lac/bronze/tickets' (FORMAT parquet, PARTITION_BY (____), ____);

-- 4. le contrôle : les tickets distincts de la source, contre les lignes du bronze
SELECT (SELECT COUNT(DISTINCT ticket_id) FROM read_csv('sources/caisse/*/tickets_*.csv')) AS source,
       (SELECT COUNT(*) FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet')) AS bronze;
```

Enregistrer, puis fermer le Bloc-notes. Le script finit par son propre contrôle : à chaque passage, il dit si chaque ticket de la source est entré une fois, et une seule.

Étape 3 — Ranger le mois · RAW

3.1 Créer le dossier du lac, puis lancer le script. `duckdb -f` exécute un fichier SQL, en mémoire : aucun entrepôt ne s'écrit, seuls les fichiers du bronze.

```
mkdir plateforme\lac\bronze
duckdb -f 01_de\sql\bronze.sql
REM Attendu : source 1527 et bronze 1527.
```

Si la commande s'arrête sur	La cause, et le geste
`IO Error: Failed to create directory "plateforme/lac/bronze/tickets"`	`plateforme\lac\bronze` n'existe pas : faire le `mkdir`, puis relancer
`IO Error: Directory "plateforme/lac/bronze/tickets" is not empty! Enable OVERWRITE option to overwrite files`	`OVERWRITE` manque à la parenthèse du `COPY`
`Conversion Error: CSV Error on Line: 2`, puis `Error when converting column "membre_id"`	le type déclaré ne tient pas la valeur : un numéro de membre, `M00073`, reste du texte. Relire la colonne nommée dans le dictionnaire
`IO Error: No files found that match the pattern "sources/caisse/*/tickets_*.csv"`	DuckDB n'est pas lancé depuis `Projet_Brulerie` : s'y placer, puis relancer

3.2 Relancer le script, pour prouver qu'il se rejoue sans rien doubler.

```
duckdb -f 01_de\sql\bronze.sql
REM Attendu : source 1527 et bronze 1527, comme au premier passage.
```

`OVERWRITE` a vidé le bronze avant de le réécrire : un bronze refait en entier ne double jamais.

3.3 Regarder le lac. `find /c "jour="` compte les dossiers des jours, comme le cours 01 comptait les exports. Un chemin qui porte un `=` se met entre guillemets : sans eux, cmd coupe l'argument à ce signe, et répond « Fichier introuvable ».

© 2026 iAlexMG. Tous droits réservés.

```
dir /b plateforme\lac\bronze\tickets | find /c "jour="
REM Attendu : 30.
dir /b "plateforme\lac\bronze\tickets\jour=2025-06-12"
REM Attendu : un seul fichier, data_0.parquet.
```

Trente dossiers pour 31 exports : le 12 juin n'a qu'un dossier, et un seul fichier, celui de son dernier export.

Étape 4 — Contrôler le bronze contre la source · RAW

Un bronze écrit n'est pas un bronze livré. Le contrôle du script dit que chaque ticket est entré une fois ; il ne dit pas que chaque jour est entier, ni que les types sont ceux qu'on a déclarés.

4.1 Ouvrir le notebook de DuckDB, depuis l'Anaconda Prompt.

```
duckdb -ui
```

Créer un notebook (+ à côté de Notebooks) et le renommer « Cours 02 ». Le notebook travaille en mémoire, comme au cours 01 : il lit les fichiers là où ils sont, et `read_parquet` lit le bronze comme `read_csv` lit la source. La colonne `jour` y revient d'elle-même, tirée du nom des dossiers.

4.2 Compter les tickets jour par jour, source contre bronze. `WITH` nomme un résultat intermédiaire, qu'on lit ensuite comme une table : il s'apprend au Cours 05 de la formation SQL avancé, `Theorie_05_SQL.md`, section 2. La jointure externe complète garde un jour qui manquerait d'un côté ; l'égalité entre deux comptes y vaut alors vide, et le `CASE` le compte comme un écart.

```
-- 1. par jour livré, les tickets distincts de la source, puis les lignes du bronze
WITH source AS (
    SELECT CAST(REGEXP_EXTRACT(filename, 'tickets_(\d{4}-\d{2}-\d{2})_', 1) AS DATE) AS jour,
           COUNT(DISTINCT ticket_id) AS tickets
    FROM read_csv('sources/caisse/*/tickets_*.csv', filename = true)
    GROUP BY jour
),
bronze AS (
    SELECT jour, COUNT(*) AS tickets
    FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet')
    GROUP BY jour
)
-- 2. un jour d'un seul côté, ou deux comptes qui diffèrent, est un écart
SELECT COUNT(*) AS jours, SUM(CASE WHEN s.tickets = b.tickets THEN 0 ELSE 1 END) AS ecarts
FROM source AS s
FULL OUTER JOIN bronze AS b ON ____;
-- Attendu : 30 jours, 0 écart.
```

4.3 Regarder le 12 juin dans le bronze.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 1. le fichier d'où viennent les tickets du 12 juin, leur nombre, le premier et le dernier
SELECT PARSE_FILENAME(filename) AS fichier, COUNT(*) AS tickets,
       MIN(horodatage) AS premier, MAX(horodatage) AS dernier
FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet')
WHERE jour = ____
GROUP BY fichier;
-- Attendu : une ligne, tickets_2025-06-12_202506130705.csv, 65 tickets, de 07:01:40 à 17:16:04.
```

65 tickets, et non 123 : le dernier export du 12, celui du 13 au matin, a fait foi.

4.4 Lire les types du bronze.

```sql
DESCRIBE SELECT * FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet');
-- Attendu : 15 colonnes : les 13 du dictionnaire, les montants en DECIMAL(10,2) ; filename ; jour en DATE.
```

Une colonne mal nommée dans `columns` ne fait pas d'erreur : DuckDB la renomme, et elle sortirait ici sous le nom que vous lui avez donné. C'est une raison de plus de lire le schéma du bronze avant de le livrer.

4.5 Additionner l'encaissé, puis relancer la même cellule deux ou trois fois (Ctrl+Entrée).

```sql
-- 1. la somme de l'encaissé, sur le bronze
SELECT SUM(encaisse) AS encaisse
FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet');
-- Attendu : 18666.38, le même à chaque fois.
```

Au cours 01, la même somme sur la source rendait presque 19 352,36, et ses derniers chiffres changeaient. L'écart, 685,98 $, est l'encaissé des 58 tickets du premier export du 12 juin, comptés deux fois. Et un `DECIMAL` s'additionne en nombres entiers de cents : l'ordre des additions ne change plus rien.

Étape 5 — Ce que le format change · RAW

Toujours dans le notebook.

5.1 Lire le schéma que porte un fichier du bronze. `parquet_schema` lit l'en-tête du fichier, sans lire ses lignes.

```sql
SELECT name, type, converted_type
FROM parquet_schema('plateforme/lac/bronze/tickets/jour=2025-06-12/*.parquet');
-- Attendu : 15 lignes : la racine, duckdb_schema, puis les 14 colonnes du fichier ; les montants en
-- INT64 et DECIMAL, horodatage en INT64 et TIMESTAMP_MICROS ; aucune ligne pour jour.
```

Le type voyage avec le fichier : qui lit le bronze n'a rien à deviner. Un montant y est rangé en nombre entier de cents, avec sa précision. Et `jour` n'est pas dans le fichier : il vit dans le nom de son dossier.

© 2026 iAlexMG. Tous droits réservés.

5.2 Lire les statistiques d'un fichier. Chaque colonne y garde sa plus petite et sa plus grande valeur : un moteur peut sauter un fichier sans l'ouvrir quand la valeur cherchée n'y tombe pas.

```
SELECT path_in_schema, row_group_num_rows, compression, stats_min, stats_max
FROM parquet_metadata('plateforme/lac/bronze/tickets/jour=2025-06-12/*.parquet')
WHERE path_in_schema IN ('ticket_id', 'horodatage');
-- Attendu : 2 lignes, 65 lignes chacune, en SNAPPY ; ticket_id de 158988 à 159052, horodatage de
-- 2025-06-12 07:01:40 à 2025-06-12 17:16:04.
```

5.3 Peser le bronze, puis la source. `read_blob` rend une ligne par fichier, avec sa taille en octets.

```
-- 1. les fichiers et les octets du bronze, puis ceux des exports de la source
SELECT (SELECT COUNT(*) FROM read_blob('plateforme/lac/bronze/tickets/*/*.parquet')) AS fichiers_bronze,
       (SELECT SUM(size) FROM read_blob('plateforme/lac/bronze/tickets/*/*.parquet')) AS octets_bronze,
       (SELECT COUNT(*) FROM read_blob('sources/caisse/*/tickets_*.csv')) AS fichiers_source,
       (SELECT SUM(size) FROM read_blob('sources/caisse/*/tickets_*.csv')) AS octets_source;
-- Attendu : 30 fichiers et un peu plus de 125 600 octets dans le bronze ; 31 fichiers et 131207
-- octets à la source.
```

Les derniers chiffres du bronze changent d'un passage du script à l'autre, de quelques dizaines d'octets : DuckDB écrit sur plusieurs fils, l'ordre des lignes dans un fichier varie, et la compression avec lui. Le contenu, lui, ne change pas. Et le bronze ne pèse presque rien de moins que la source, c'est la leçon : chaque fichier Parquet porte son schéma et ses statistiques, et une soixantaine de tickets se compressent mal. Mesuré, le même mois rangé dans un seul fichier pèse environ 35 000 octets, le quart. Parquet gagne sur le volume ; sur de petits fichiers, il gagne surtout ses types. Boréal, au même cours, le mesure sur 843 fichiers : c'est le piège des petits fichiers.

5.4 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans l'Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, toujours dans `Projet_Brulerie`.

Étape 6 — Versionner · RAW

6.1 Valider le `.gitignore` et le script. `git -C 01_de` lance Git dans le dépôt sans quitter `Projet_Brulerie` ; le lac, hors du dépôt, n'y entre pas.

```
git -C 01_de add .gitignore sql
git -C 01_de commit -m "Cours 02 : les tickets en bronze, un dossier par jour"
REM Attendu : 2 files changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

© 2026 iAlexMG. Tous droits réservés.

- Le bronze range ce qui est livré dans un format qui se lit vite, avec des types déclarés : un montant en `DECIMAL` compte au cent près, et ne change pas d'une exécution à l'autre.
- Un dossier par jour livré : le chemin porte le jour, et d'un jour exporté deux fois, le dernier export fait foi.
- Un bronze se contrôle contre sa source, jour par jour, avant de servir ; sur de petits fichiers, Parquet gagne surtout ses types.

© 2026 iAlexMG. Tous droits réservés.